

Cómo Piensan los Modelos

Una intuición sólida sobre qué es un LLM, por qué funciona y dónde falla — sin una sola fórmula.

1. ¿Qué es un LLM y cómo genera texto?
2. Probabilidad y alucinaciones
3. Fortalezas reales y fallos sistemáticos
4. Qué diferencia a los modelos entre sí
5. Cómo leer un benchmark

Navega con   o los botones de abajo

1

TEMA 1

¿Qué es un LLM y cómo genera texto?

Del chef experimentado a la predicción token a token.

Un LLM es como un chef con 20 años de oficio

ANALOGÍA · EL CHEF

Un chef experimentado no sigue una receta para improvisar. Ha cocinado tanto que cuando le pides *"algo con pollo, limón y jengibre"* lo crea sin haberlo visto antes: **combina patrones** que absorbió durante años.

Un **LLM** (Large Language Model) hace lo análogo. Leyó miles de millones de páginas — libros, código, conversaciones. No las memorizó: ajustó millones de valores internos llamados **pesos** para capturar los patrones del lenguaje.

EN RESUMEN

Un LLM no busca en una base de datos ni sigue reglas escritas por humanos. **Predice el texto más probable** dada toda la conversación, usando patrones estadísticos aprendidos de una cantidad enorme de texto humano.

A diferencia del autocompletado del celular —que solo mira las últimas palabras— el LLM mantiene coherencia con todo el texto previo y genera texto nuevo que nunca existió.

Tokens: la unidad mínima de trabajo

El modelo no trabaja con letras ni con palabras completas, sino con **tokens**: fragmentos de texto. Como las fichas de Scrabble — las palabras se arman combinando fichas, pero las fichas no siempre coinciden con sílabas naturales.

Texto de entrada:

"La inteligencia artificial es fascinante"

7 tokens:

La ·intelig encia ·artific ial ·es ·fascinante · = espacio inicial

Tokenización aproximada — la división exacta varía según el modelo.

¿Por qué importa?

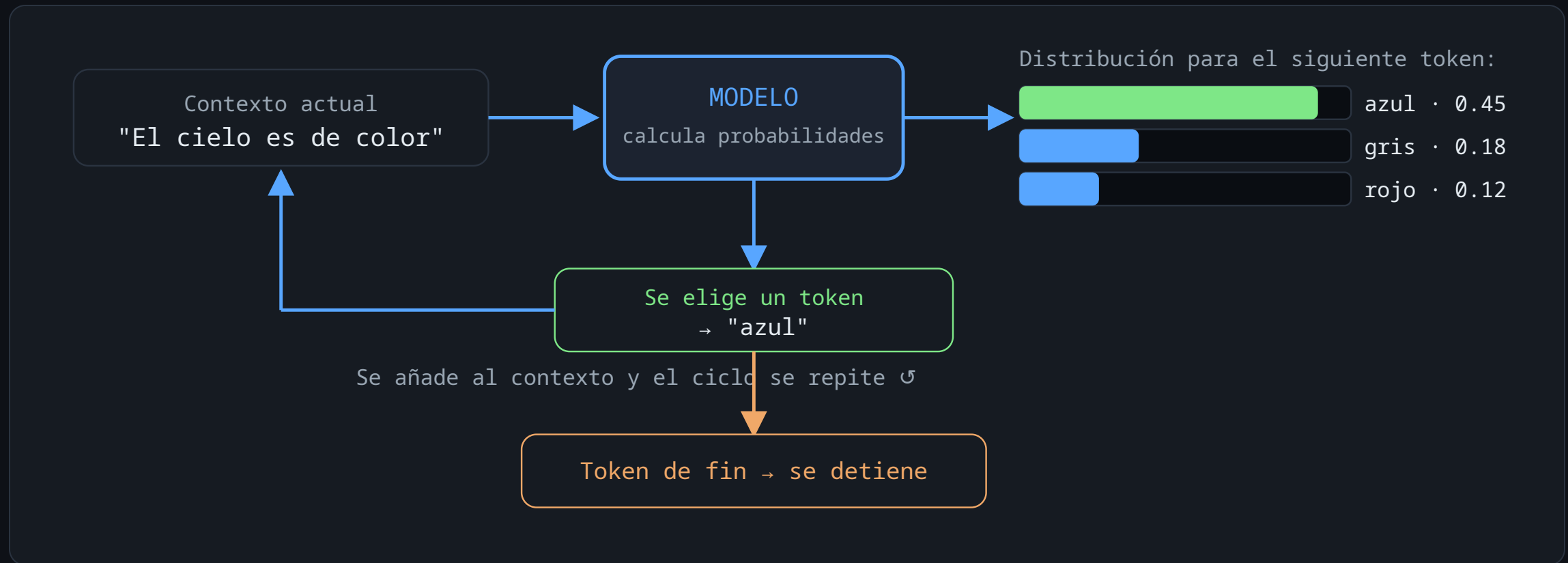
- Los modelos **cobran por tokens** (los que envías + los que generan).
- Hay un **límite máximo** por conversación (la ventana de contexto, Módulo 2).

El detalle curioso

Que el modelo cuente mal las letras de una palabra se explica aquí: nunca ve "inteligencia" como una unidad — la ve como trozos.

El texto se construye un token a la vez

Cada nuevo token se añade al contexto para generar el siguiente. Es un ciclo **autoregresivo**: lo que parece una respuesta instantánea son cientos de predicciones encadenadas.



Generación autoregresiva: predecir · elegir · añadir · repetir.

LA IDEA CLAVE

El modelo no "piensa" la respuesta completa y luego la escribe. La construye token a token, **sin poder revisar lo que ya generó**.

El Transformer, sin matemáticas

La arquitectura base se llama **Transformer** (2017). Dos mecanismos clave bastan para la intuición:

Atención (Attention)

En una conversación larga, tu cerebro recupera un detalle dicho hace 20 minutos cuando hace falta. La atención hace lo equivalente: al predecir el siguiente token, el modelo **"mira hacia atrás"** sobre todo el texto y decide **cuánto peso** darle a cada palabra anterior.

La clave: ese peso **no es fijo**. Se recalcula para cada token y según el contexto. La misma palabra puede ser crucial en una frase e irrelevante en la siguiente.

Por eso un modelo resuelve ambigüedades que el autocompletado no puede:

```
"Dejé el libro en la mesa  
porque era muy pesado."
```

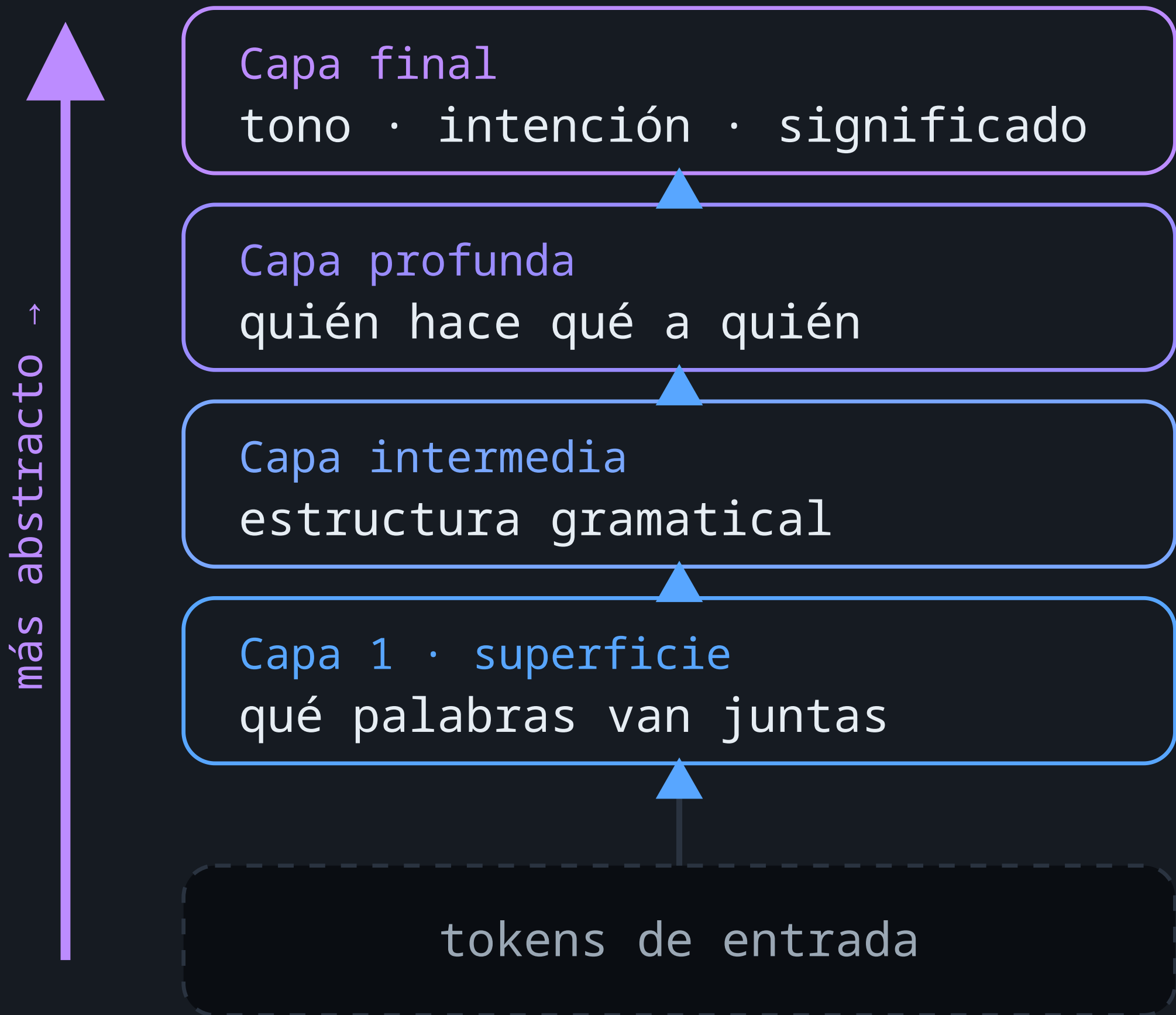
```
¿Qué era pesado?  
El modelo conecta "era pesado"  
con "libro" (no con "mesa").
```

Para predecir bien, la atención liga *era pesado* con *libro* y casi ignora *mesa*. Eso es resolver la referencia, sin reglas gramaticales escritas a mano.

Y no mira un solo aspecto a la vez: en paralelo, distintas "cabezas" de atención rastrean cosas diferentes —una la concordancia, otra el sujeto, otra el tema general—. De ahí sale la coherencia en textos largos.

Capas de transformación

Muchas capas apiladas. Cada una recibe la salida de la anterior y la refina un poco más: las primeras detectan patrones simples (qué palabras van juntas); las profundas, relaciones abstractas (quién hace qué a quién, el tono, la intención).



Cada capa refina la representación de la anterior.

DE DÓNDE VIENE EL NOMBRE "GPT"

GPT = Generative Pre-trained Transformer. No es un producto: son las tres ideas de este módulo juntas — **G**enera texto token a token (1.3), fue **P**re-entrenado a gran escala (4.2) y usa la arquitectura **T**ransformer (esta slide). Casi todos los modelos modernos —Claude, Gemini, Llama incluidos— comparten esta base. Ojo: "GPT" (la arquitectura) no es lo mismo que "ChatGPT" o "GPT-5.5" (los productos de OpenAI).

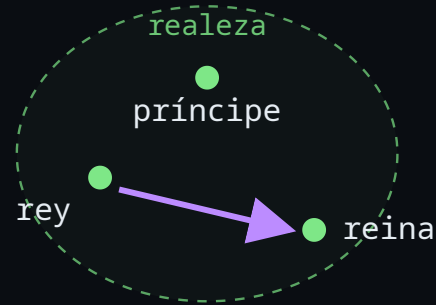
El modelo **no "entiende"** el texto como un humano: manipula representaciones de las relaciones entre tokens. Que eso produzca resultados aparentemente comprensivos es sorprendente y real, pero el mecanismo subyacente es estadístico.

El espacio vectorial: un mapa de significado

¿Qué son esas "representaciones de las relaciones entre tokens"? Cada token se convierte en un **punto en un espacio de muchas dimensiones** — imposible de dibujar entero, pero la intuición se ve bien en un mapa plano: los tokens con significado parecido quedan **cerca**, y las relaciones entre ellos se vuelven **direcciones consistentes**. Las capas profundas refinan ese mapa, de lo superficial a lo abstracto.

Mapa de significado

cerca = parecido · flecha = relación



misma dirección = misma relación (género)



Capas: superficie → abstracto

Capa 1 · superficie
qué palabras
tienden a ir juntas

Capa intermedia
estructura
gramatical

Capa más profunda
quién hace
qué a quién

Capa final · abstracto
tono, intención,
significado global

más profundo

Mapa plano ilustrativo de un espacio real de cientos o miles de dimensiones.

La intuición que importa: el modelo no guarda definiciones de las palabras. Guarda *posiciones* y *direcciones* — y de la geometría entre esos puntos emergen las relaciones de significado, sin que nadie las haya escrito a mano.

2

TEMA 2

Probabilidad y alucinaciones

El dial de la creatividad y por qué los modelos inventan con confianza.

Temperatura: el dial de la creatividad

Al elegir el siguiente token, la **temperatura** controla cuánta aleatoriedad se inyecta en esa elección.

ANALOGÍA · EL DIAL DEL HORNO

Es como el dial de temperatura de un horno, pero aplicado a la **personalidad**: bajo = siempre la misma respuesta predecible; alto = cada vez sale algo distinto.

Distribución para el siguiente token:

"azul" → 45% "gris" → 18% "rojo" → 12% "verde" → 8% ...

Temperatura = 0 (frío, determinista)

- Siempre "azul". Predecible y repetible.
- Como un funcionario que sigue el reglamento al pie de la letra.

Temperatura = 0.7 (tibia, balanceada – el default habitual)

- "azul" casi siempre, pero a veces "gris" o "rojo".
- Como un buen profesional: confiable, con criterio propio.

Temperatura = 1.5 (alta, creativa)

- Las probabilidades se aplanan. Cualquier opción puede salir.
- Como un artista improvisando: a veces brillante, a veces incoherente.

Precisión (código, datos) → temperatura baja. Creatividad (redacción libre, brainstorming) → alta. En interfaces como claude.ai el modelo elige el valor apropiado según el contexto.

Por qué los modelos inventan cosas

EL PROBLEMA #1 PARA UN USUARIO NUEVO

Las **alucinaciones** son respuestas que suenan plausibles y se expresan con confianza, pero son incorrectas, inventadas o directamente falsas.

ANALOGÍA · EL QUE NUNCA DICE "NO SÉ"

Ese compañero que ante cualquier pregunta responde con seguridad. A veces acierta porque lo sabe; pero cuando no sabe, improvisa algo que suena razonable. El LLM tiene ese comportamiento *por diseño*.

La causa técnica

No accede a una base de datos de hechos. Genera el token más probable. Si el dato está poco representado en su entrenamiento, no tiene mecanismo para decir "no lo tengo": genera lo que estadísticamente parece correcto.

Pregunta: "¿Resultado del partido
Copiapó vs O'Higgins el
14 de marzo de 2019?"

El modelo no tiene ese dato.
Pero sabe que los resultados
tienen la forma "X ganó Y-Z".

Genera:
"Copiapó ganó 2-1 en el
Estadio Municipal."

→ Inventado. Presentado con
total confianza.

Tipos de alucinación y cómo reconocerlas

TIPO	DESCRIPCIÓN	EJEMPLO
Factual	Inventa datos concretos: fechas, cifras, nombres	"Fundada en 1987" (falso pero preciso)
Citación	Inventa referencias que no existen	Papers con título y autores plausibles pero inexistentes
Confabulación	Mezcla hechos reales con inventados de forma coherente	Biografía real con eventos ficticios intercalados
Razonamiento	Pasos intermedios incorrectos en una deducción	Resultado correcto por camino equivocado, o viceversa
Recencia	Datos desactualizados presentados como actuales	"El CEO es [nombre de hace 3 años]"

Señales de alerta

- Datos muy específicos sobre un tema oscuro o poco documentado.
- Misma seguridad sobre lo verificable y lo no verificable.
- Menciona fuentes sin que se las hayas pedido.

Regla práctica

Cuanto más **específico y verificable** sea el dato, más vale confirmarlo por fuera. Para razonamiento, síntesis y generación de texto la tasa de error es baja. Para hechos puntuales, siempre verifica.

3

TEMA 3

Fortalezas reales y fallos sistemáticos

Qué confiar al modelo — y qué nunca darle por sentado.

Lo que los modelos hacen excepcionalmente bien

Síntesis y comprensión

Extraer puntos clave, resumir y responder sobre un documento largo — incluso muy técnico.

Transformación de formato

Texto no estructurado → JSON, Markdown, CSV. Reformatear tablas, extraer campos. Muy fiable.

Generación de código

Código funcional con alta frecuencia en lenguajes populares: vio millones de repositorios.

Reescritura y tono

Técnico ↔ accesible, formal ↔ conversacional. Alta calidad.

Razonar sobre contexto dado

"Dado este contrato, ¿qué cláusula aplica?" Funciona muy bien si la respuesta está en el contexto provisto.

Generación de variantes

5 nombres de función, 3 versiones de un párrafo, casos de prueba a demanda. Muy valioso a diario.

Dónde fallan de forma predecible

Matemática y cálculo exacto

No calculan: **predicen** el texto que parece el resultado. A veces coincide; otras no.

$17 \times 83 + 44$

→ Puede acertar, pero no porque calculó.

7931×6847

→ Aquí el error sube mucho.

Solución: usa la **ejecución de código** integrada en lugar de confiar en el texto.

Conteo sobre listas / caracteres

"¿Cuántas 'a' hay aquí?" es sorprendentemente difícil: procesa tokens, no caracteres.

Conocimiento reciente o muy específico

Todo lo posterior a su fecha de corte es invisible. Eventos poco documentados → más riesgo de alucinación.

Muchas condiciones a la vez

A más restricciones simultáneas, más probable que cumpla unas y "olvide" otras.

Documentos muy largos

La atención a partes lejanas se degrada: los detalles del inicio se "diluyen" hacia el final.

¿Cuánto confiar en una respuesta?

SI LA TAREA REQUIERE...	CONFIANZA RECOMENDADA
Síntesis de texto provisto	Alta
Generación de código (lenguaje popular)	Alta con revisión
Razonamiento sobre reglas explícitas dadas	Alta
Hechos históricos bien documentados	Media
Hechos específicos recientes	Baja — verificar
Cálculo numérico exacto	Baja — usar código
Conteo de caracteres o elementos	Muy baja — verificar siempre
Eventos oscuros o poco documentados	Muy baja — alta prob. de alucinación

PRINCIPIO CLAVE

El modelo es un **colaborador muy capaz, no un oráculo**. Trata sus respuestas como un borrador inteligente, no como fuente de verdad. Tu rol es dar dirección y verificar lo que importa.

4

TEMA 4

Qué diferencia a los modelos entre sí

Parámetros, pretraining, fine-tuning y RLHF.

Tamaño: parámetros y lo que significan

Los modelos se describen por su número de **parámetros** — los valores ajustados durante el entrenamiento. Se miden en miles de millones: 7B, 13B, 70B, 405B. Pensados como las *conexiones entre neuronas* de un cerebro: más conexiones no garantizan más inteligencia, pero permiten representar relaciones más complejas.

¿Más parámetros = mejor? Sí, con matices:

- Modelos más grandes requieren más hardware y son más lentos y costosos.
- Un modelo pequeño bien entrenado puede superar a uno grande entrenado descuidadamente en tareas específicas.
- Para la mayoría de usos cotidianos, un modelo **mid-size** es más que suficiente y mucho más económico que el más grande.

Entrenamiento base (pretraining)

La fase más costosa y lenta: se expone al modelo a cantidades masivas de texto y se le entrena para **predecir el siguiente token** una y otra vez, ajustando los pesos en cada iteración.

Es un único objetivo repetido a escala astronómica: **adivinar la siguiente palabra**. De tanto hacerlo, el modelo termina absorbiendo gramática, hechos y estructura del mundo — todo de forma indirecta, como efecto secundario de predecir bien.

Es la fase que consume meses de cómputo y la mayor parte del costo de crear un modelo.

Un paso de pretraining:

1. Corpus: "La fotosíntesis convierte luz en [____]"
2. Modelo predice → "calor"
3. Token real → "energía"
4. Se ajustan los pesos para reducir ese error
5. Repetir × miles de millones

Al terminar sabe mucho, pero no es útil aún: puede continuar cualquier texto, pero no responder preguntas de forma estructurada ni seguir instrucciones.

Fine-tuning: especialización post-fábrica

ANALOGÍA · LA RESIDENCIA MÉDICA

Un médico recién graduado (modelo base) tiene formación amplia. En la residencia de cardiología (fine-tuning) no empieza de cero: **profundiza sobre la base que ya tiene**. Al terminar es el mismo médico, mucho más experto en ese dominio.

Instruction tuning

Miles de ejemplos pregunta → respuesta. Transforma un "predicador de texto" en un "asistente".

Domain fine-tuning

Textos de un dominio (legal, médico, técnico). Mejora en ese campo concreto.

Format fine-tuning

Responder siempre en un formato específico (JSON, tablas, etiquetas).

El fine-tuning **no crea conocimiento de la nada**: si el modelo base no conoce un dominio, no lo genera. Solo refuerza patrones existentes o ajusta comportamiento y formato.

RLHF: enseñar preferencias humanas

Reinforcement Learning from Human Feedback — la técnica que hace a los modelos útiles, seguros y alineados con lo que los humanos prefieren. En tres pasos:

1 · Recopilar

Varias respuestas a la misma pregunta; humanos dicen cuál prefieren. Dataset de comparaciones.

2 · Reward model

Un modelo auxiliar aprende a predecir qué gusta más. Actúa como juez automático.

3 · Ajustar el LLM

El juez puntúa; se ajustan los pesos del LLM para subir la puntuación. Ciclo iterativo.

Por esto Claude o GPT responden de forma útil, declinan peticiones problemáticas y tienen un "carácter". Cada empresa usa sus propios datos de preferencia → personalidades distintas.

Tabla comparativa — actualizada a junio 2026

MODELO	EMPRESA	FORTALEZA / DATO 2026	CUÁNDO USARLO
Claude Opus 4.8	Anthropic	Mejor overall hoy; líder en código (SWE-bench Verified 88,6%) y escritura larga	Tareas complejas, código, flujos autónomos
Claude Sonnet 4.6	Anthropic	~80% del código de Opus a una fracción del costo	Uso cotidiano, alto volumen
GPT-5.5	OpenAI	Generalista muy fuerte, ecosistema amplio	Uso general, multimodal
Gemini 3.1 Pro	Google	Mejor precio/rendimiento; GPQA Diamond 94,3%; contexto enorme	Razonamiento, documentos masivos
Grok 4.3	xAI	Ventana de contexto de hasta ~2M tokens	Documentos muy largos
DeepSeek V4	DeepSeek	Open weights, calidad casi-frontera muy económica	Alternativa económica, autohospedaje
Llama 4	Meta	Open source, muy rápido, contexto largo	Privacidad, control local
Kimi K2.6 / Qwen 3	Moonshot / Alibaba	Top de los modelos open weights por capacidad	Open source de alto nivel

El panorama cambia cada pocos meses — esta tabla es de **junio 2026**. Dato contraintuitivo de 2026: los modelos de "razonamiento" tienden a **alucinar más, no menos** (todos >10% en mediciones recientes). Los benchmarks son punto de partida; **la prueba en tu caso de uso es siempre más informativa.**

5

TEMA 5

Cómo leer un benchmark

Qué miden, qué ocultan, y la única prueba que de verdad importa.

Qué mide (y qué no) un benchmark

Un **benchmark** es un conjunto de pruebas estandarizadas con respuestas conocidas, para medir el rendimiento en condiciones controladas y comparables.

ANALOGÍA · EL EXAMEN PISA

Como las pruebas PISA o el examen de admisión: miden ciertas habilidades de forma estandarizada y permiten comparar. Pero quien saca 100 puede ser un profesional mediocre, y quien sacó 80, brillante. **El examen mide lo que el examen mide.**

Qué SÍ mide

- Rendimiento en ese tipo de preguntas, en ese momento
- Comparación relativa entre modelos, en igualdad
- Progreso de un modelo entre versiones

Qué NO mide

- Rendimiento en *tu* tarea específica
- Calidad en conversación larga
- Fiabilidad en dominios especializados o en español
- Lo que importa para tu flujo real

Benchmarks comunes y su utilidad real

BENCHMARK	QUÉ MIDE	UTILIDAD PRÁCTICA
MMLU	Conocimiento general en 57 dominios	Media — amplitud general
HumanEval / SWE-bench	Código funcional, resolución de bugs	Alta para quienes programan
MATH / AIME	Problemas matemáticos de competencia	Alta si necesitas razonamiento matemático
GPQA	Preguntas nivel PhD en ciencias	Razonamiento profundo; poco relevante a diario
MT-Bench	Conversación multi-turno, instrucciones	Media — capacidad de asistente
Arena (LMSYS)	Preferencias humanas en conversación libre	Alta — satisfacción de usuarios reales

BENCHMARK HACKING

Los laboratorios pueden optimizar sus modelos para los benchmarks públicos que conocen, inflando resultados. Un modelo con **92% en MMLU** no es necesariamente mejor para redactar propuestas comerciales que uno con **88%**. Varios de estos benchmarks clásicos (MMLU, GSM8K, HumanEval) hoy se consideran **saturados o contaminados** → siguiente slide.

¿Y si el modelo ya vio el examen?

El problema más serio de los benchmarks en 2026: las preguntas de los tests públicos **se filtran a los datos de entrenamiento** (crawls web, GitHub, foros). Un modelo puede puntuar alto simplemente porque **memorizó las respuestas** — no porque razone mejor. Y como los laboratorios conocen los tests famosos, pueden optimizar para ellos. Hoy la contaminación se asume **por defecto**, no como excepción.

La respuesta: benchmarks resistentes a contaminación

Diseñados para que sus preguntas *no puedan* estar en el entrenamiento. Son los que los laboratorios serios reportan hoy:

BENCHMARK	CÓMO EVITA LA CONTAMINACIÓN	QUÉ MIDE
LiveBench	Se renueva con problemas nuevos cada mes	Capacidad general, siempre fresca
LiveCodeBench	Solo problemas publicados <i>después</i> del corte del modelo	Programación real, sin memorizar
FrontierMath	Problemas inéditos escritos por matemáticos, nunca publicados	Matemática de nivel investigación
ARC-AGI 2	Set de evaluación privado (holdout)	Razonamiento abstracto novedoso
HLE (Humanity's Last Exam)	Holdout privado; 3.000 preguntas de 1.000+ expertos	Conocimiento experto multidisciplinar
SWE-bench Verified · GPQA Diamond	Versiones revisadas/"a prueba de Google", validadas por expertos	Código real · ciencia nivel PhD

PREGUNTA QUE VALE ORO

Ante cualquier puntaje, pregúntate: **¿este benchmark está diseñado para que el modelo no haya podido memorizarlo?** Si la respuesta es no, el número dice mucho menos de lo que parece.

La prueba definitiva: tu caso de uso

La forma más fiable de elegir: toma **5–10 tareas reales** que necesitas habitualmente, ejecútalas en los 2–3 modelos candidatos y evalúa.

1

¿Cuál resuelve mejor el problema **sin instrucciones adicionales**?

2

¿Cuál produce el **formato** que necesitas?

3

¿Cuál comete **menos errores** en tu dominio?

4

¿Cuál conviene en **velocidad y costo** para tu volumen?

Este proceso toma una hora y **vale más que cualquier tabla de benchmarks.**

Los 5 temas, de un vistazo

#	TEMA	IDEA CENTRAL	PARA LLEVAR
1	LLM y generación	Predice el texto más probable, un token a la vez (autoregresivo)	No busca en una BD: combina patrones, como un chef
2	Probabilidad y alucinaciones	La temperatura regula el azar; sin azar de hechos, inventa con confianza	A más específico y verificable el dato, más hay que confirmarlo
3	Fortalezas y fallos	Excelente en síntesis, formato y código; flojo en cálculo y conteo	Colaborador capaz, no oráculo: da dirección y verifica
4	Diferencias entre modelos	Parámetros + pretraining + fine-tuning + RLHF definen el carácter	Mid-size suele bastar; el panorama cambia cada pocos meses
5	Leer un benchmark	Mide condiciones controladas, no tu tarea; ojo al benchmark hacking y la contaminación	La prueba real: 5–10 tareas tuyas en 2–3 modelos